

Encontro 3: Estruturas de Controle (if, else, switch)

Sumário

Encontro 3: Estruturas de Controle (if, else, switch).....	1
1. Introdução às Estruturas de Controle.....	2
2. A Estrutura Condicional `if`	2
3. A Estrutura `else`: O Caminho Alternativo	2
4. Encadeando Condições com `else if`	3
5. A Estrutura `switch` (ou `escolha-caso`).....	4
6. Operadores Lógicos e Relacionais em Estruturas de Controle.....	5
7. Exercícios Práticos	5
Exercício 1: Calculadora de IMC	5
Exercício 2: Menu de Opções	6
Exercício 3: Verificador de Ano Bissexto	7

1. Introdução às Estruturas de Controle

As estruturas de controle são a espinha dorsal da lógica de programação. Elas permitem que nossos programas tomem decisões e sigam caminhos diferentes com base em condições específicas. Sem elas, nossos algoritmos seriam apenas uma sequência linear de instruções. Neste encontro, vamos explorar as estruturas de decisão `if`, `else` e `switch`, que são fundamentais para criar programas dinâmicos e inteligentes.

2. A Estrutura Condicional `if`

A estrutura `if` (se) é a forma mais básica de controle de decisão. Ela avalia uma condição e, se essa condição for verdadeira, executa um bloco de código. Se a condição for falsa, o bloco de código é simplesmente ignorado.

Sintaxe em Pseudocódigo:

```
se (condição) entao
    // Bloco de código a ser executado se a condição for verdadeira
fimse
```

Exemplo: Verificador de Maioridade

```
start
declare age: integer = 20
if (age >= 18) then
    write "Você é maior de idade."
endif
end
```

Figura 1: Exemplo de estrutura condicional if básica em pseudocódigo.

O algoritmo apresentado na figura 1 demonstra como verificar se uma pessoa é maior de idade usando uma condição simples e executar uma ação apenas quando a condição é verdadeira. Neste exemplo, como a idade (20) é maior ou igual a 18, a mensagem "Você é maior de idade." será exibida.

3. A Estrutura `else`: O Caminho Alternativo

Frequentemente, queremos que o programa faça algo quando a condição do `if` é falsa. Para isso, usamos a cláusula `else` (senão). O bloco de código dentro do `else` é executado somente se a condição do `if` for falsa.

Sintaxe em Pseudocódigo:

```
se (condição) entao
    // Bloco de código para condição verdadeira
senao
    // Bloco de código para condição falsa
fimse
```

Exemplo: Par ou Ímpar

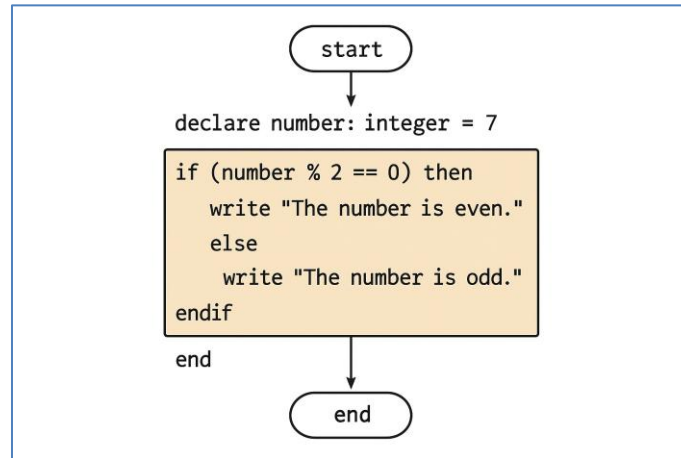


Figura 2: Exemplo de estrutura if-else para verificar se um número é par ou ímpar.

A figura 2 mostra como usar o operador módulo (%) e como o programa executa diferentes blocos de código dependendo da condição.

4. Encadeando Condições com `else if`

Para testar múltiplas condições em sequência, podemos usar o `else if` (senão se). Isso nos permite criar uma cadeia de verificações. O programa testa cada condição na ordem em que aparecem e executa o bloco de código da primeira condição que for verdadeira. Se nenhuma for verdadeira, o bloco `else final` (se existir) é executado.

Exemplo: Classificação por Idade

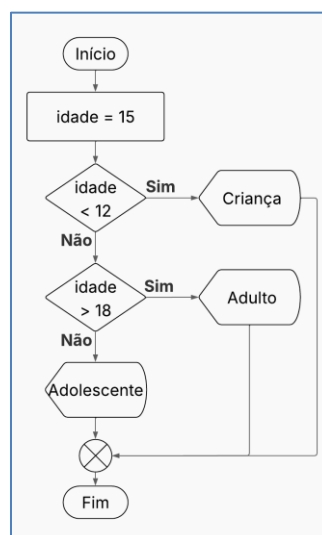


Figura 3: Exemplo de estrutura else-if para classificação por faixa etária.

A figura 3 demonstra como testar múltiplas condições em sequência e como o programa executa apenas o primeiro bloco cuja condição seja verdadeira.

5. A Estrutura `switch` (ou `escolha-caso`)

A estrutura `switch` é uma alternativa ao uso de múltiplos `if-else` `if` quando precisamos comparar uma única variável com vários valores possíveis. Ela torna o código mais limpo e legível em certas situações.

Sintaxe em Pseudocódigo:

```
escolha (variavel)
  caso valor1:
    // Bloco de código para valor1
  pare
  caso valor2:
    // Bloco de código para valor2
  pare
  outrocaso:
    // Bloco de código se nenhum caso corresponder
fimescolha
```

O comando `pare` (ou `break` em muitas linguagens) é crucial. Ele impede que a execução continue nos casos seguintes após uma correspondência ser encontrada.

Exemplo: Dia da Semana

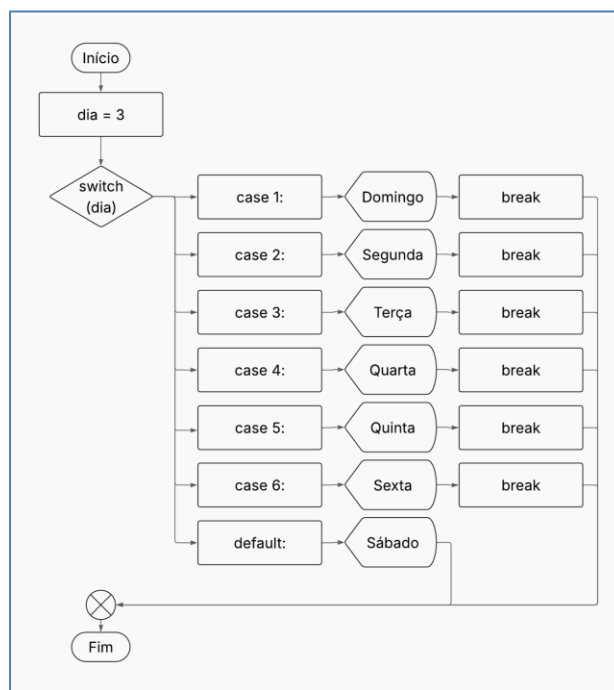


Figura 4: Exemplo de estrutura switch/escolha-caso para determinar o nome do dia da semana.

A figura 4 mostra como comparar uma variável com múltiplos valores e a importância do comando 'pare' para evitar execução em cascata.

6. Operadores Lógicos e Relacionais em Estruturas de Controle

O poder das estruturas de controle vem da sua combinação com operadores lógicos e relacionais. Eles nos permitem criar condições complexas e precisas.

- Operadores Relacionais (`==`, `!=`, `>`, `<`, `>=`, `<=`): Usados para comparar valores.
- Operadores Lógicos (`E`, `OU`, `NÃO`): Usados para combinar múltiplas condições.

Exemplo: Verificador de Acesso

```
inicio
  declare idade: inteiro = 25
  declare possuiConvite: logico = verdadeiro
  se (idade >= 18 E possuiConvite == verdadeiro) entao
    escreva "Acesso permitido."
  senao
    escreva "Acesso negado."
  fimse
fim
```

Neste exemplo, o acesso só é permitido se a pessoa for maior de 18 anos **E** possuir um convite. Ambas as condições precisam ser verdadeiras.

7. Exercícios Práticos

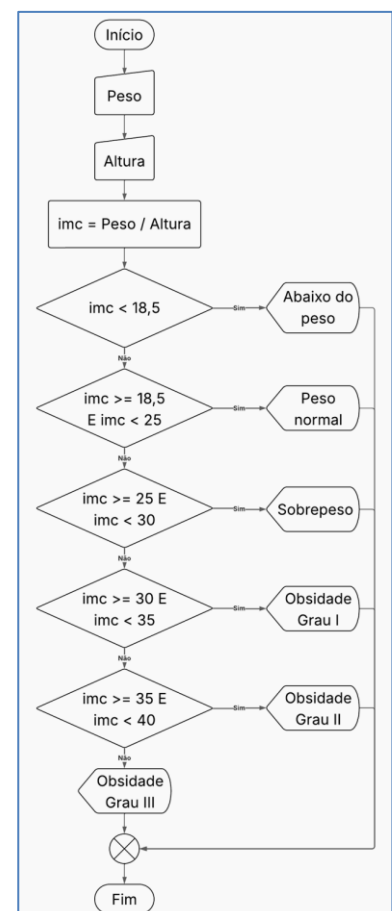
Exercício 1: Calculadora de IMC

Crie um algoritmo que leia o peso (em kg) e a altura (em metros) de uma pessoa e calcule o seu Índice de Massa Corporal (IMC). A fórmula é: $IMC = \text{peso} / (\text{altura} * \text{altura})$. Em seguida, classifique o resultado de acordo com a tabela:

- $IMC < 18.5$: Abaixo do peso
- $18.5 \leq IMC < 25$: Peso normal
- $25 \leq IMC < 30$: Sobrepeso
- $IMC \geq 30$: Obesidade

Figura 5: Fluxograma para calculadora de IMC.

Este exercício demonstra o uso de múltiplas estruturas condicionais encadeadas para classificar o resultado do cálculo em diferentes categorias de saúde.



Exercício 2: Menu de Opções

Desenvolva um algoritmo que exiba um menu com 4 opções:

1. Ver Saldo,
2. Fazer Saque,
3. Fazer Depósito,
4. Sair.

Utilize a estrutura `switch` para exibir uma mensagem correspondente à opção escolhida pelo usuário.

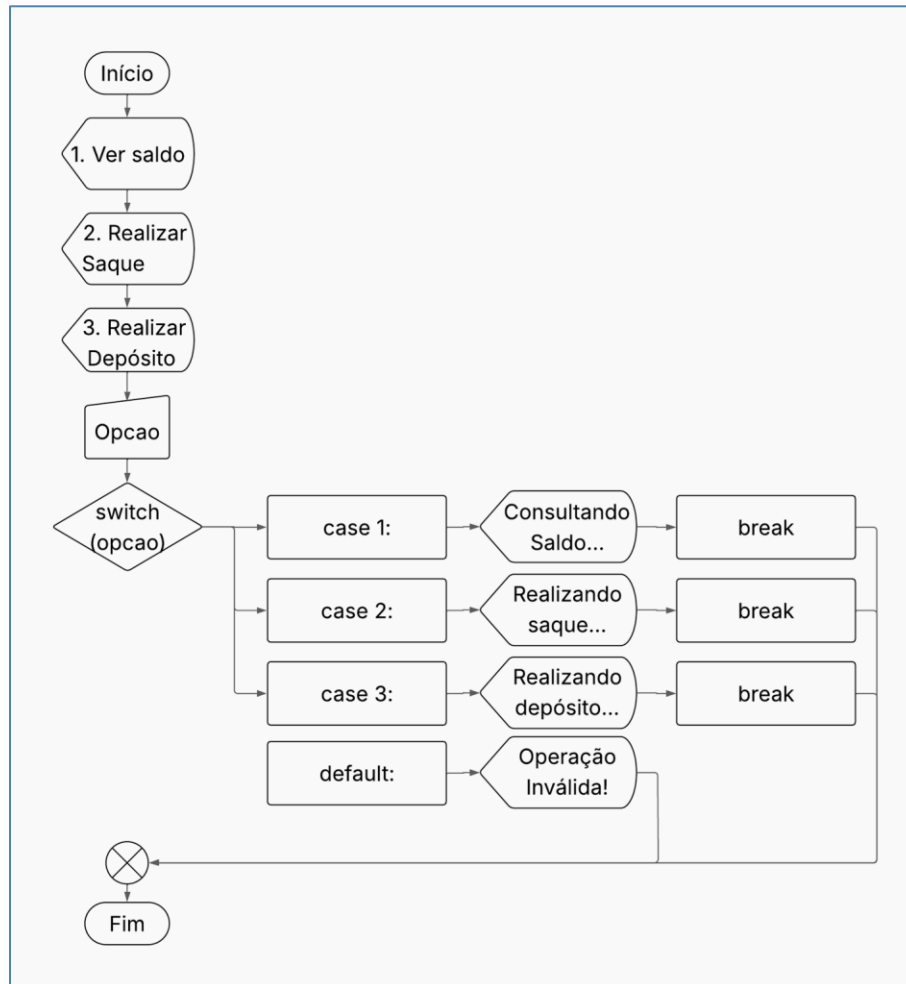


Figura 6: Fluxograma para menu de opções bancárias usando estrutura switch.

Este exercício mostra como criar um sistema de menu interativo onde cada opção executa uma ação específica.

Exercício 3: Verificador de Ano Bissexto

Faça um algoritmo que leia um ano e determine se ele é bissexto. Um ano é bissexto se for divisível por 4, mas não por 100, exceto se for divisível por 400. (Ex: 2000 e 2024 são bissextos; 1900 e 2021 não são). Use operadores lógicos para construir a condição.

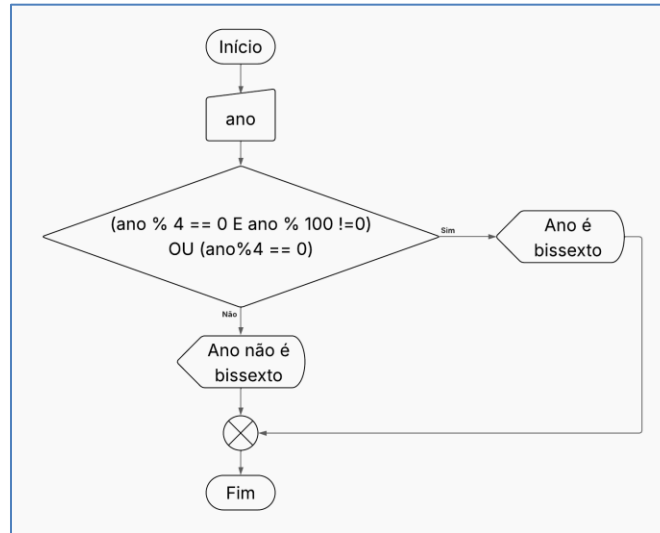


Figura 7: Fluxograma para verificar ano bissexto.

Este exercício demonstra o uso de operadores lógicos complexos (E, OU) para criar uma condição que verifica múltiplas regras simultaneamente.